Deep Learning for CPU Workload Prediction and Intelligent Process Scheduling

Saloni Kushwaha
Rajiv Gandhi Institute of Petroleum Technology, India
Research Intern, Indian Institute of Information Technology Allahabad, India
24it3045@rgipt.ac.in

***Abstract*—Modern operating systems rely on traditional heuristic-based CPU scheduling algorithms—such as Round Robin, Shortest Job First, and Completely Fair Scheduler—that operate without any predictive knowledge of future workload patterns. As computing environments grow increasingly dynamic, heterogeneous, and bursty, these static policies fail to efficiently utilize processor resources, leading to increased latency, energy waste, and degraded throughput. In this work, we propose DeepSched, a deep learning-based framework for CPU workload prediction and intelligent scheduling assistance. The proposed system will employ a hybrid CNN-LSTM architecture augmented with a multi-head attention mechanism to predict future CPU utilization patterns from historical workload traces. The predicted workload profiles will be consumed by a scheduling advisor module that recommends scheduling decisions aimed at minimizing average waiting time, turnaround time, and energy consumption. A Reinforcement Learning (RL) based scheduling agent using Proximal Policy Optimization (PPO) will be trained to learn an adaptive policy through interaction with a simulated OS environment. Experiments are planned on real-world CPU trace datasets (Google Cluster Trace and Alibaba Cluster Trace) to evaluate the proposed approach against classical scheduling algorithms and prior deep learning baselines.**

***Index Terms*—CPU Scheduling, Workload Prediction, Deep Learning, LSTM, Attention Mechanism, Reinforcement Learning, Operating Systems, Resource Management, Process Scheduling, Time-Series Forecasting.**

I. Introduction

THE CPU scheduler is one of the most critical components of any modern operating system. It determines which process receives processor time, for how long, and in what order—decisions that directly affect system responsiveness, throughput, and energy efficiency. Classical scheduling algorithms such as First-Come-First-Served (FCFS), Round Robin (RR), Shortest Job First (SJF), and the Linux Completely Fair Scheduler (CFS) are designed around simple heuristics or fairness policies [10]. While these algorithms have served well for decades, they operate reactively—making decisions based solely on the current system state—and have no mechanism to anticipate future workload behaviour.

Contemporary computing workloads, particularly in cloud environments, data centres, and mobile systems, are highly dynamic and exhibit complex temporal patterns [8]. Workload bursts, periodic task arrivals, and heterogeneous process mixes make it difficult for static scheduling policies to maintain high utilisation while keeping latency low. Suboptimal scheduling decisions lead to increased context-switching overhead, processor idle time, higher energy consumption, and degraded quality of service for latency-sensitive applications.

Recent advances in deep learning—particularly in sequence modelling with Recurrent Neural Networks (RNNs) [4], Transformers [11], and Reinforcement Learning (RL) [7]—offer a promising avenue to address these limitations. Deep learning models can capture complex non-linear temporal dependencies in workload traces and produce accurate short-horizon forecasts. RL agents can learn scheduling policies that optimise long-term reward signals such as throughput and energy efficiency, adapting dynamically to changing workloads.

This work presents **DeepSched**, a system that applies deep learning to the problem of CPU workload prediction and scheduling. The framework consists of two interacting modules: (i) a Workload Prediction Module (WPM) based on a CNN-LSTM-Attention architecture, and (ii) a Scheduling Advisor Module (SAM) that uses predicted workload profiles to make smarter scheduling decisions, culminating in a PPO-based RL agent.

The key contributions of this work are:

- A proposed CNN-LSTM-Attention workload prediction model to be trained on real-world CPU trace datasets, capturing both local and long-range temporal dependencies.
- A scheduling advisor module that will translate predicted workload profiles into actionable scheduling hints to reduce average waiting time and turnaround time.
- A reinforcement learning-based scheduling agent (PPO) to be trained in a simulated OS environment, learning adaptive policies beyond what hand-crafted heuristics can achieve.
- A comprehensive evaluation plan against classical and deep learning scheduling baselines on real CPU trace benchmarks.

Unlike DeepRM [5] and related reinforcement learning schedulers that make decisions solely from the current system state, DeepSched incorporates explicit workload forecasting through a CNN-LSTM-Attention prediction module. This enables the scheduler to leverage anticipated future system conditions, allowing proactive scheduling decisions rather than purely reactive policies. Furthermore, the proposed framework separates workload forecasting and scheduling into dedicated modules, allowing explicit incorporation of future workload information into scheduling decisions.

The remainder of this proposal is structured as follows. Section II reviews related work. Section III describes the proposed methodology and architecture. Section IV covers the datasets used. Section V concludes with future directions.

II. Related Work

A. Classical CPU Scheduling

Traditional CPU scheduling is extensively studied in operating systems literature [10]. FCFS suffers from the convoy effect; SJF minimises average waiting time but requires prior knowledge of burst times; Round Robin offers fairness at the cost of high context-switch overhead. The Linux CFS uses a red-black tree ordered by virtual runtime to approximate ideal processor sharing. Multilevel feedback queues add adaptivity but are governed by manually tuned parameters. None of these algorithms learn from historical workload patterns.

B. Workload Prediction

Time-series forecasting for system workload prediction has been explored using ARIMA [1], exponential smoothing, and regression models. Deep learning approaches have demonstrated superior performance: LSTMs [4] capture long-range temporal dependencies in workload traces, while Temporal Convolutional Networks (TCNs) offer parallelisable sequence modelling. Transformer-based forecasting models [11] such as Informer and Autoformer have recently achieved state-of-the-art results on long-sequence prediction tasks relevant to CPU utilisation forecasting.

C. Deep Learning for Operating Systems

The application of machine learning to OS-level resource management has gained traction. Paragon [2] applied collaborative filtering to heterogeneous workload classification. Quasar [3] extended this to resource allocation. DeepRM [5] proposed a neural network-based job scheduler using policy gradients, demonstrating that RL agents can outperform heuristic schedulers in data centre settings. Decima [6] applied graph neural networks to cluster job scheduling. Our work focuses specifically on single-node CPU scheduling with a practical CNN-LSTM-Attention prediction front-end.

D. Reinforcement Learning for Scheduling

RL has been applied to scheduling in data centres [5], network packet scheduling, and memory management. Policy gradient methods (REINFORCE, PPO, A3C) are commonly used. The key challenge is defining an appropriate reward function that reflects scheduling objectives without introducing bias. Recent work has explored multi-objective RL for jointly optimising latency and energy in heterogeneous processors, which is directly relevant to our approach.

III. Proposed Methodology

The proposed **DeepSched** framework consists of two modules operating in a pipeline: the Workload Prediction Module (WPM) and the Scheduling Advisor Module (SAM). Figure 1 illustrates the overall system architecture.

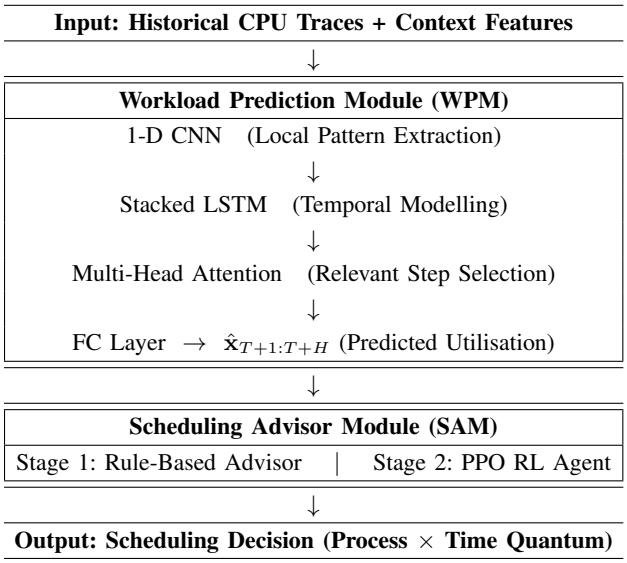

Fig. 1: DeepSched Architecture. Historical CPU traces flow through the WPM (CNN → LSTM → Attention) to forecast future utilisation. The SAM converts predictions into scheduling decisions via a rule-based advisor (Stage 1) or a PPO RL agent (Stage 2).

A. System Architecture

DeepSched follows a two-stage pipeline comprising a Workload Prediction Module (WPM) and a Scheduling Advisor Module (SAM). Historical CPU utilisation traces and contextual system features are first processed by the WPM to generate short-horizon workload forecasts. These forecasts are subsequently utilised by the SAM to guide both rule-based and reinforcement learning-based scheduling decisions, as illustrated in Figure 1.

B. Problem Formulation

1) Workload Prediction: Let $\mathbf{x} = \{x_1, x_2, \ldots, x_T\}$ denote a time-series of CPU utilisation measurements sampled at discrete intervals, where $x_t \in [0, 1]$ represents the normalised CPU utilisation at time t. The workload prediction problem is to learn a function f_θ such that:

$$\hat{\mathbf{x}}_{T+1:T+H} = f_\theta(x_{T-L+1:T}) \tag{1}$$

where L is the look-back window, H is the prediction horizon, and θ are learnable parameters. The objective is to minimise the mean squared prediction error:

$$\mathcal{L}_{\text{pred}} = \frac{1}{H} \sum_{h=1}^{H} (x_{T+h} - \hat{x}_{T+h})^2 \tag{2}$$

2) Scheduling Optimisation: Let $\mathcal{P} = \{p_1, p_2, \ldots, p_N\}$ be a set of N processes in the ready queue, each characterised by arrival time a_i, estimated burst time b_i, and priority π_i. The scheduling objective is to find an ordering σ of $\mathcal{P}$ that minimises:

$$J(\sigma) = \alpha \cdot \overline{W}(\sigma) + \beta \cdot \overline{T}(\sigma) + \gamma \cdot E(\sigma) \tag{3}$$

where $\overline{W}$ is the average waiting time, $\overline{T}$ is the average turnaround time, E is total energy consumption, and α, β, γ are weighting coefficients.

3) RL Scheduling Agent: The scheduling problem is formulated as a Markov Decision Process (MDP):

- **State** s_t: Current ready queue composition, predicted CPU utilisation $\hat{x}_{t:t+H}$, and recent scheduling history.
- **Action** a_t: Selection of the next process to execute and its allocated time quantum.
- **Reward** r_t: Negative weighted sum of waiting time and energy at step t.
- **Policy** $\pi_\theta(a_t|s_t)$: A neural network mapping states to action probabilities.

The agent is trained to maximise the expected cumulative discounted reward:

$$\max_\theta \ \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right] \tag{4}$$

C. Workload Prediction Module (WPM)

1) Convolutional Feature Extraction: A 1-D convolutional front-end extracts local temporal patterns from the input sequence before feeding into the LSTM, improving sensitivity to short-duration workload spikes and reducing noise. Beyond raw CPU utilisation, the WPM ingests contextual features including number of active processes, memory utilisation, I/O wait percentage, and time-of-day encoding, concatenated as multivariate inputs.

2) LSTM with Multi-Head Attention: The LSTM layers capture sequential dependencies in CPU utilisation time series:

$$\mathbf{h}_t, \mathbf{c}_t = \text{LSTM}(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1}) \tag{5}$$

The hidden states $\{\mathbf{h}_1, \ldots, \mathbf{h}_T\}$ are then passed through a multi-head self-attention layer [11] to focus on the most relevant past time steps:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{6}$$

where Q, K, V are query, key, and value matrices derived from the LSTM hidden states. The attended representations are projected through a fully connected layer to produce the H-step ahead prediction $\hat{\mathbf{x}}_{T+1:T+H}$.

D. Scheduling Advisor Module (SAM)

1) Rule-Based Advisor (Stage 1): In the first stage, the WPM's predictions are used by a rule-based advisor that adjusts scheduling parameters. If predicted utilisation exceeds a threshold τ_h, the advisor reduces the time quantum for interactive processes to improve responsiveness. If predicted utilisation is low, it pre-schedules background batch processes to improve throughput.

2) Reinforcement Learning Agent (Stage 2): In the second stage, a Proximal Policy Optimisation (PPO) [9] agent replaces the rule-based advisor. The agent receives the state s_t—which includes the WPM's predictions—and outputs a scheduling decision. The neural policy network uses a two-layer MLP with ReLU activations, trained in a custom simulated OS environment (built using OpenAI Gym) where processes arrive according to distributions fitted to real workload traces.

3) Planned Implementation: The WPM will be implemented in PyTorch using stacked LSTM layers with multi-head attention. The PPO agent will use an MLP policy network trained via the Adam optimiser. A custom OpenAI Gym environment will simulate process arrivals drawn from distributions fitted to real workload traces, enabling controlled end-to-end training and evaluation.

E. Evaluation Plan

DeepSched will be compared against FCFS, SJF, Round Robin ($q = 20$ ms), CFS, ARIMA with rule-based scheduling, vanilla LSTM (no attention), and DeepRM [5]. Prediction performance will be measured by MAE, RMSE, and MAPE. Scheduling performance will be assessed using Average Waiting Time (AWT), Average Turnaround Time (ATT), CPU Utilisation (%), Throughput, and an energy proxy via a linear CPU frequency-power model.

IV. Dataset

A. Google Cluster Trace Dataset

The Google Cluster Workload Traces [8] provide fine-grained CPU utilisation measurements from a production cluster of approximately 12,500 machines over 29 days. We extract per-machine CPU utilisation time series at 5-minute intervals to train and evaluate the WPM.

B. Alibaba Cluster Trace

The Alibaba Cluster Trace (2018) contains CPU and memory utilisation records from a 1,300-machine cluster, providing additional diversity in workload patterns including batch and online service mixes.

The Google and Alibaba traces were selected because they represent large-scale production workloads exhibiting significant temporal variability, making them suitable benchmarks for evaluating predictive scheduling approaches.

C. Synthetic Scheduling Traces

For the RL scheduling simulator, synthetic process arrival traces are generated according to Poisson arrival processes with CPU burst times drawn from distributions fitted to the Google trace data. This allows controlled experimentation with varying load intensities.

V. Conclusion and Future Work

This proposal presents DeepSched, a deep learning-based framework for CPU workload prediction and intelligent process scheduling. By combining a CNN-LSTM-Attention prediction module with a PPO reinforcement learning scheduling

agent, DeepSched moves beyond reactive heuristic schedulers to enable proactive, data-driven scheduling decisions. The framework integrates convolutional feature extraction, recurrent sequence modelling, attention mechanisms, and reinforcement learning to address a practically important and academically rich problem at the intersection of operating systems and machine learning.

Future work will explore Transformer-based prediction backbones (e.g., Informer, PatchTST) for longer-horizon forecasting, multi-objective RL reward formulations that explicitly trade off energy and latency, and deployment studies on real Linux kernel scheduling via eBPF hooks. Extension to multi-core and heterogeneous CPU architectures also presents a natural next step.

REFERENCES

[1] George E. P. Box, Gwilym M. Jenkins, Gregory C. Reinsel, and Greta M. Ljung. *Time Series Analysis: Forecasting and Control*. Wiley, Hoboken, NJ, 5th edition, 2015.

[2] Christina Delimitrou and Christos Kozyrakis. Paragon: QoS-aware scheduling for heterogeneous datacenters. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 77–88. ACM, 2013.

[3] Christina Delimitrou and Christos Kozyrakis. Quasar: Resource-efficient and QoS-aware cluster management. In *Proceedings of the Nineteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 127–144. ACM, 2014.

[4] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[5] Hongzi Mao, Mohammad Alizadeh, Ishai Menache, and Srikanth Kandula. Resource management with deep reinforcement learning. In *Proceedings of the Fifteenth ACM Workshop on Hot Topics in Networks (HotNets)*, pages 50–56. ACM, 2016.

[6] Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. Learning scheduling algorithms for data processing clusters. In *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM)*, pages 270–288. ACM, 2019.

[7] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[8] Charles Reiss, Alexey Tumanov, Gregory R. Ganger, Randy H. Katz, and Michael A. Kozuch. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In *Proceedings of the Third ACM Symposium on Cloud Computing (SoCC)*, pages 1–13. ACM, 2012.

[9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[10] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. Wiley, Hoboken, NJ, 10th edition, 2018.

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30. Curran Associates, 2017.